

Deep Learning & GenAI — Exam-Focus Deep Dive

Deep Learning & GenAI — Exam-Focus Deep Dive

Purpose. A self-study companion built around the TA's end-sem guidance: explain the high-priority topics from first principles (intuition → math → diagram → worked numerical), and drill the **5–8 mark** problems the model paper does not contain. Pair with file 02 (cheatsheet) for in-exam lookup and files 03/04 for testing yourself.

What the TA said to expect. The paper mixes intuitive and mathematical questions. MCQs/short answers test intuition (with the occasional small matrix-multiply or parameter count); the **long-answer section is where math lives — proofs and derivations**. Priority topics flagged: vanilla seq2seq & the Markov assumption; perplexity & empirical entropy (know how to compute them); pre-training vs fine-tuning, and **last-layer vs full fine-tuning** trade-offs (performance + compute); the three transformer families; zero-/few-shot prompting; **parameter counts** (incl. positional-encoding parameters); masking methods; the **RLHF pipeline**; and the **Bradley-Terry derivation** plus PEFT trade-offs. Everything is from the second half of the course.

How to use this document

Each major topic has four parts:

- **Intuition** — the one idea you must not forget.
- **Math** — the equations, written so you can reproduce them.
- **Diagram** — a picture of the data flow.
- **Worked numerical** — a full solution at exam length and difficulty.

The  marker flags a worked numerical sized for the long-answer section (5–8 marks). Every worked numerical follows the same four steps so you can copy the method under exam pressure:

1. **Formula to use** — write the equation first, and say what each symbol means.
2. **Given** — list the numbers from the question.
3. **Step-by-step substitution** — plug in and show the actual arithmetic, line by line.
4. **Answer + interpretation** — box the result and say what it means.

When you revise, cover the steps and reproduce them from the Formula line — that is exactly what earns marks in the long-answer section.

Part 1 — Language modelling: Markov, seq2seq, perplexity & entropy

1.1 The Markov assumption and n-gram models

Intuition. A language model needs $P(w_1, \dots, w_T)$. By the chain rule this factorises exactly:

$$P(w_1 \dots T) = \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1})$$

The problem: $P(w_t \mid w_1 \dots w_{t-1})$ conditions on an unbounded history, which we can never count reliably. The **Markov assumption** truncates the history to the last $n-1$ tokens:

$$P(w_t \mid w_1, \dots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}, \dots, w_{t-1})$$

- $n = 1$ (unigram): no context, $P(w_t)$.
- $n = 2$ (bigram): one token of context, $P(w_t \mid w_{t-1})$.
- $n = 3$ (trigram): two tokens, $P(w_t \mid w_{t-2}, w_{t-1})$.

Why it matters / its limit. The assumption makes counting tractable but throws away long-range dependencies ("The *keys* that I left on the table ... *are* mine" — agreement across many words). This bottleneck is exactly what RNNs (unbounded hidden state) and later attention (direct access to every token) were built to fix.

Maximum-likelihood estimate (MLE) for an n-gram:

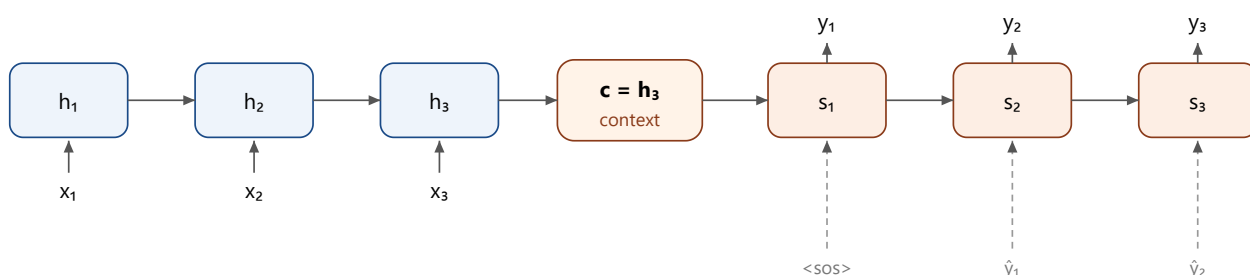
$$P(w_t \mid w_{t-n+1} \dots t-1) = \text{count}(w_{t-n+1} \dots w_t) / \text{count}(w_{t-n+1} \dots w_{t-1})$$

1.2 Vanilla sequence-to-sequence (RNN encoder–decoder)

Intuition. Seq2Seq maps an input sequence to an output sequence of possibly different length (translation, summarisation). An **encoder** RNN reads the whole input and compresses it into a single **context vector** c (its last hidden state). A **decoder** RNN then generates the output one token at a time, conditioned on c and on what it has produced so far.

ENCODER

DECODER



The entire input is squeezed through one fixed-size vector c — the "bottleneck".

Figure 1. Vanilla RNN seq2seq. The context vector $c = h_3$ is the only channel from input to output — the bottleneck that attention later removes.

The bottleneck problem. All information about a 40-word source sentence must fit in one fixed vector c . Performance degrades sharply with input length. **Attention** (Bahdanau et al.) fixes this by letting the

decoder look back at *all* encoder states $h_1 \dots h_T$ at every step, forming a *different* context c_t per output token:

$$\begin{aligned} e_{\{t,j\}} &= v^T \tanh(W_s s_{\{t-1\}} + W_h h_j) && \text{(alignment score)} \\ \alpha_{\{t,j\}} &= \text{softmax}_j(e_{\{t,j\}}) && \text{(attention weights)} \\ c_t &= \sum_j \alpha_{\{t,j\}} h_j && \text{(per-step context)} \end{aligned}$$

This is the conceptual seed of self-attention in transformers.

1.3 Empirical entropy, cross-entropy, and perplexity

These three are one idea viewed three ways. Know how to write *and compute* each — the TA called this out explicitly.

Empirical entropy $H(p)$ measures the average uncertainty (in bits, if $\log_2$) of the *true* distribution p of the data:

$$H(p) = - \sum_x p(x) \log_2 p(x)$$

Cross-entropy $H(p, q)$ measures the average number of bits needed to encode data drawn from p when we use our *model* q :

$$H(p, q) = - \sum_x p(x) \log_2 q(x) \geq H(p) \quad (\text{equality iff } q = p)$$

In practice we don't know p ; we estimate cross-entropy directly on a held-out corpus of N tokens using the model's own probabilities:

$$H = - (1/N) \sum_{t=1}^N \log_2 q(w_t \mid \text{context})$$

Perplexity is just the exponentiated cross-entropy — the “effective branching factor”, i.e. how many equally-likely choices the model is effectively deciding among at each step:

$$\begin{aligned} \text{PPL} &= 2^{\{H\}} && \text{(base-2 logs)} && \text{or} && \text{PPL} = \exp(H) && \text{(natural logs)} \\ \text{equivalently} &&& \text{PPL} &= && \left(\prod_{t=1}^N q(w_t \mid \text{context}) \right)^{-1/N} \end{aligned}$$

Lower perplexity = better model. A model that is perfectly certain and correct has $\text{PPL} = 1$; a uniform model over a vocabulary of size V has $\text{PPL} = V$.

 Worked numerical (≈6 marks). Cross-entropy and perplexity.

A bigram model scores a 4-token test sequence (after < sos >). Its predicted probabilities for the actual next tokens are:

step t	token	$q(w_t w_{\{t-1\}})$
1	the	1/2
2	cat	1/4
3	sat	1/8
4	down	1/8

Compute (a) the per-token cross-entropy in bits and (b) the perplexity.

Solution.

Formula to use.

Cross-entropy: $H = - (1/N) \cdot \sum_{t=1}^N \log_2 q(w_t)$

Perplexity: $PPL = 2^H$

where N = number of tokens scored, and $q(w_t)$ = the probability the model assigned to the token that actually occurred. We use $\log_2$, so H comes out in bits.

Given. $N = 4$ tokens, with probabilities $q = [1/2, 1/4, 1/8, 1/8]$.

Step 1 — turn each probability into $\log_2$ (its “bits”). Useful fact: $\log_2(1/2^k) = -k$.

$$\log_2(1/2) = \log_2(2^{-1}) = -1$$

$$\log_2(1/4) = \log_2(2^{-2}) = -2$$

$$\log_2(1/8) = \log_2(2^{-3}) = -3$$

$$\log_2(1/8) = \log_2(2^{-3}) = -3$$

Step 2 — add the logs.

$$\sum \log_2 q = (-1) + (-2) + (-3) + (-3) = -9$$

Step 3 — average over N and negate → this is H (part a).

$$H = - (1/N) \cdot (\sum \log_2 q) = - (1/4) \cdot (-9) = 9/4 = 2.25 \text{ bits/token}$$

Step 4 — raise 2 to the power H → perplexity (part b).

$$PPL = 2^H = 2^{2.25} = 2^2 \cdot 2^{0.25} = 4 \times 1.189 = 4.757$$

Answer. (a) $H = 2.25$ bits/token. (b) $PPL \approx 4.76$. **Interpretation.** A perplexity of ~ 4.76 means the model is, on average, as uncertain as if it were guessing uniformly among ~ 4.76 equally-likely words at each step (lower is better). **Sanity check.** Perplexity also equals the inverse geometric mean of the probabilities: $(\frac{1}{2} \cdot \frac{1}{4} \cdot \frac{1}{8} \cdot \frac{1}{8})^{-1/4} = (1/512)^{-1/4} = 512^{1/4} = 4.757$. ✓ Same answer.

Worked numerical (≈5 marks). Bigram MLE with add-k smoothing.

Corpus (with sentence markers): $\langle s \rangle$ I am Sam $\langle /s \rangle$ $\langle s \rangle$ Sam I am $\langle /s \rangle$ $\langle s \rangle$ I do not like eggs $\langle /s \rangle$ Vocabulary $V = \{I, am, Sam, do, not, like, eggs, \langle /s \rangle\}$, so $|V| = 8$.

a. Unsmoothed: $P(am \mid I)$. (b) Add-1 (Laplace) smoothed: $P(am \mid I)$. (c) Why is smoothing needed?

Solution.

Formula to use.

Unsmoothed (MLE): $P(w_2 \mid w_1) = \text{count}(w_1 w_2) / \text{count}(w_1)$

Add-k smoothed: $P(w_2 \mid w_1) = (\text{count}(w_1 w_2) + k) / (\text{count}(w_1) + k \cdot |V|)$

Here $\text{count}(w_1 w_2)$ = how many times the pair " $w_1 w_2$ " occurs, $\text{count}(w_1)$ = how many times " w_1 " occurs, $|V|$ = vocabulary size, and k = smoothing constant ($k=1$ is Laplace).

Step 1 — collect the counts from the corpus.

$\text{count}(I) = 3$ (the word "I" appears three times)

$\text{count}(I \text{ am}) = 2$ (the pairs "I am" in "I am Sam" and "Sam I am"; the 3rd "I" is followed by "do")

$|V| = 8$

Step 2 — (a) unsmoothed. Substitute into the MLE formula.

$P(am \mid I) = \text{count}(I \text{ am}) / \text{count}(I) = 2 / 3 \approx 0.667$

Step 3 — (b) add-1. Substitute into the add-k formula with $k = 1$.

$$\begin{aligned} P(am \mid I) &= (\text{count}(I \text{ am}) + 1) / (\text{count}(I) + 1 \cdot |V|) \\ &= (2 + 1) / (3 + 1 \cdot 8) \\ &= 3 / 11 \approx 0.273 \end{aligned}$$

(c) Why smoothing? Under the unsmoothed MLE, any bigram never seen in training gets probability 0; one such zero makes the whole sentence probability 0 and perplexity infinite. Add-k shifts a little mass to unseen pairs so novel sequences get a non-zero probability. (Notice the trade-off: smoothing pulled $P(am \mid I)$ from 0.667 down to 0.273 — add-k can over-smooth when $|V|$ is large.)

Part 2 — Recurrent cells and their parameter counts

Intuition. A vanilla RNN's gradient must flow through many `tanh` multiplications, so it vanishes (or explodes) over long sequences. **LSTM/GRU** add gated, additive paths for the cell/hidden state so gradients can travel far — the "gradient highway".

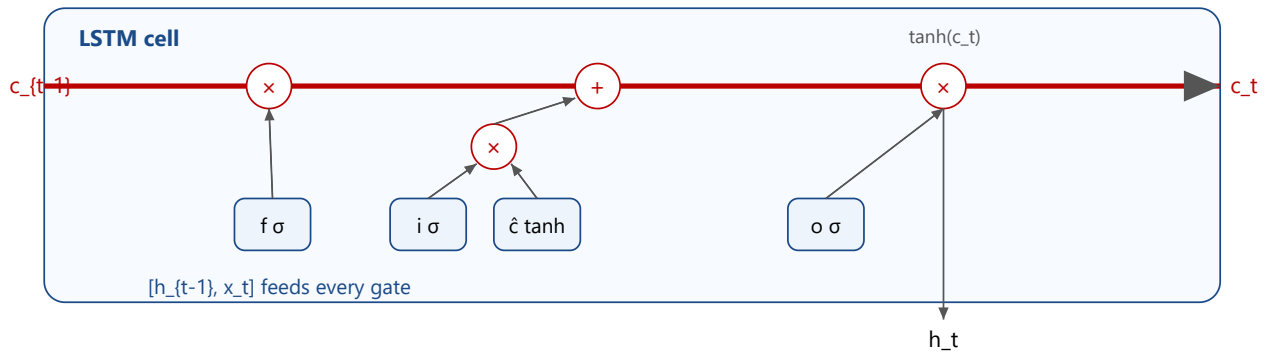


Figure 2. LSTM cell. The red cell-state line is nearly multiplication-free: when the forget gate $f \approx 1$, $\partial c_t / \partial c_{t-1} \approx 1$, so gradients do not vanish.

Equations (memorise).

LSTM

$$\begin{aligned} f &= \sigma(W_f \cdot [h_{t-1}, x_t]) \\ i &= \sigma(W_i \cdot [h_{t-1}, x_t]) \\ \hat{c} &= \tanh(W_c \cdot [h_{t-1}, x_t]) \\ c_t &= f \odot c_{t-1} + i \odot \hat{c} \\ o &= \sigma(W_o \cdot [h_{t-1}, x_t]) \\ h_t &= o \odot \tanh(c_t) \end{aligned}$$

GRU

$$\begin{aligned} r &= \sigma(W_r \cdot [h_{t-1}, x_t]) \\ z &= \sigma(W_z \cdot [h_{t-1}, x_t]) \\ \hat{h} &= \tanh(W \cdot [r \odot h_{t-1}, x_t]) \\ h_t &= (1-z) \odot h_{t-1} + z \odot \hat{h} \end{aligned}$$

Parameter-count rule. A gate is an affine map from the concatenation $[h_{t-1}, x_t]$ (size $n + m$) to a vector of size n , i.e. a weight $n \times (n+m)$ plus bias $n \rightarrow n(n+m+1)$ parameters. Count the gates:

RNN : 1 affine $\rightarrow n(m + n + 1)$
 GRU : 3 affines $\rightarrow 3 \cdot n(m + n + 1)$
 LSTM : 4 affines $\rightarrow 4 \cdot n(m + n + 1)$

Worked numerical (≈5 marks). Recurrent parameter counts.

Input embedding size $m = 100$, hidden size $n = 128$. Give the trainable-parameter count of one RNN, one GRU, and one LSTM layer.

Solution.

Formula to use. Each gate is one affine map from the concatenated input $[h_{t-1}, x_t]$ (length $n+m$) to a length- n vector, so it has $n \times (n+m)$ weights plus n biases:

```
params per gate =  $n \cdot (n + m) + n = n \cdot (m + n + 1)$ 
RNN has 1 gate  $\rightarrow 1 \cdot n(m+n+1)$ 
GRU has 3 gates  $\rightarrow 3 \cdot n(m+n+1)$ 
LSTM has 4 gates  $\rightarrow 4 \cdot n(m+n+1)$ 
```

Step 1 — compute the base block $n(m + n + 1)$. With $m = 100$, $n = 128$:

```
 $m + n + 1 = 100 + 128 + 1 = 229$ 
base block =  $n \cdot 229 = 128 \times 229 = 29,312$ 
```

Step 2 — multiply by the number of gates.

```
RNN =  $1 \times 29,312 = 29,312$ 
GRU =  $3 \times 29,312 = 87,936$ 
LSTM =  $4 \times 29,312 = 117,248$ 
```

Answer. RNN 29,312 · GRU 87,936 · LSTM 117,248. **Interpretation.** GRU costs 3× and LSTM 4× a plain RNN per layer — that is exactly the price of their extra gates.

Part 3 — Self-attention and masking

3.1 Scaled dot-product self-attention

Intuition. Each token emits a **query** (what it is looking for), a **key** (what it offers), and a **value** (the content it passes on). A token's output is the average of all values, weighted by how well its query matches each key. Unlike an RNN, every token reaches every other token in *one* step — no recurrence, fully parallel.

```
Attention(Q, K, V) = softmax(  $QK^T / \sqrt{d_k}$  ) V
```

- QK^T gives raw similarity scores between every query and every key.
- $\sqrt{d_k}$ keeps the dot products from growing with dimension (which would saturate the softmax into near one-hot, killing gradients).
- softmax turns scores into weights that sum to 1 per row.
- Multiplying by V produces the weighted sum. The output of position l is $a_l = \sum_t \alpha_{l,t} v_t$ (weighted sum of **values**, not queries or keys — a classic MCQ trap).

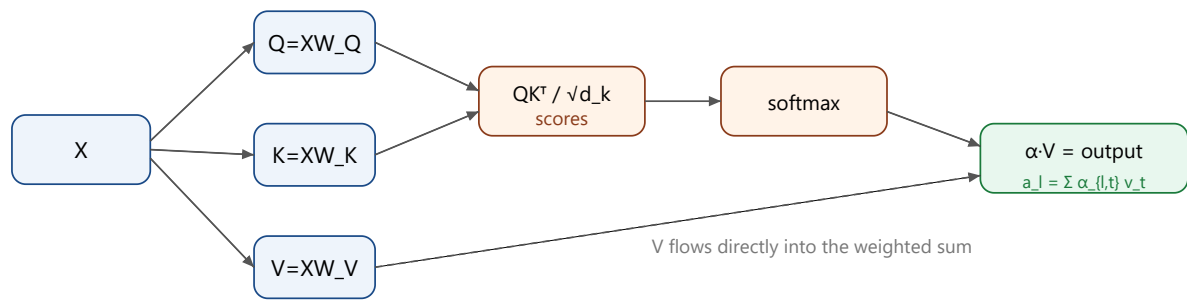


Figure 3. Self-attention dataflow: $X \rightarrow (Q, K, V) \rightarrow \text{scaled scores} \rightarrow \text{softmax weights} \rightarrow \text{weighted sum of values}$.

 Worked numerical (≈6 marks). One attention head by hand.

Two tokens, $d_k = 2$. Given (rows = tokens):

$$Q = \begin{bmatrix} [1, 0], \\ [0, 1] \end{bmatrix}, \quad K = \begin{bmatrix} [1, 0], \\ [1, 0] \end{bmatrix}, \quad V = \begin{bmatrix} [1, 0], \\ [0, 1] \end{bmatrix}$$

Compute the attention output for **query row 1**.

Solution.

Formula to use. $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$. For one query q against keys k_j and values v_j :

1. $\text{score}_j = q \cdot k_j$ (dot product of query with each key)
2. $\text{scaled}_j = \text{score}_j / \sqrt{d_k}$ (divide by $\sqrt{d_k}$ to control magnitude)
3. $\alpha_j = \text{softmax}(\text{scaled}_j) = e^{\{\text{scaled}_j\}} / \sum_i e^{\{\text{scaled}_i\}}$
4. $\text{output} = \sum_j \alpha_j \cdot v_j$ (weighted sum of the value vectors)

Recall the dot product $[a, b] \cdot [c, d] = a \cdot c + b \cdot d$, and $\sqrt{d_k} = \sqrt{2} \approx 1.414$.

Step 1 — scores: dot the query $q_1 = [1, 0]$ with each key.

$$\begin{aligned} \text{score}_1 &= q_1 \cdot k_1 = [1, 0] \cdot [1, 0] = 1 \cdot 1 + 0 \cdot 0 = 1 \\ \text{score}_2 &= q_1 \cdot k_2 = [1, 0] \cdot [1, 0] = 1 \cdot 1 + 0 \cdot 0 = 1 \quad \rightarrow \text{scores} = [1, 1] \end{aligned}$$

Step 2 — scale by $\sqrt{d_k}$.

$$\text{scaled} = [1/1.414, 1/1.414] = [0.707, 0.707]$$

Step 3 — softmax. The two values are equal, so each gets half:

$$\alpha = [e^{\{0.707\}} / (e^{\{0.707\}} + e^{\{0.707\}}), \dots] = [0.5, 0.5]$$

Step 4 — weighted sum of values.

$$\text{output} = 0.5 \cdot v_1 + 0.5 \cdot v_2 = 0.5 \cdot [1, 0] + 0.5 \cdot [0, 1] = [0.5, 0.5]$$

Answer. $[0.5, 0.5]$ — with equal keys the query splits attention evenly.

Follow-up — why we scale, and how peaking arises. Keep $q_1 = [1, 0]$, v as above, but make the keys distinct: $k_1 = [1, 0]$, $k_2 = [0, 1]$.

$$\begin{aligned} \text{Step 1 } \text{scores} &= [q_1 \cdot k_1, q_1 \cdot k_2] = [1, 0] \\ \text{Step 2 } \text{scaled} &= [1/1.414, 0] = [0.707, 0] \\ \text{Step 3 } \alpha &= [e^{\{0.707\}}, e^{\{0\}}] / (e^{\{0.707\}} + e^{\{0\}}) \\ &= [2.028, 1] / 3.028 = [0.670, 0.330] \\ \text{Step 4 } \text{output} &= 0.670 \cdot [1, 0] + 0.330 \cdot [0, 1] = [0.670, 0.330] \end{aligned}$$

Now attention leans toward token 1. The bigger the gap between scores, the peakier α becomes — and dividing by $\sqrt{d_k}$ keeps that gap from blowing up as the dimension grows (which would saturate the softmax and kill gradients).

3.2 Masking methods (be thorough — TA flagged this)

There are **three** distinct uses of masking; do not confuse them.

(a) Causal / look-ahead mask (decoder self-attention). Preserves the autoregressive property: position 1 may not attend to any future position $t > 1$. Implemented by setting scores to $-\infty$ before softmax, so those weights become 0:

$$e_{\{1,t\}} = -\infty \quad \text{for all } t > 1 \quad \Rightarrow \quad \text{softmax sends } \alpha_{\{1,t\}} \rightarrow 0$$

Causal mask (4 tokens)

query $\downarrow$, key $\rightarrow$

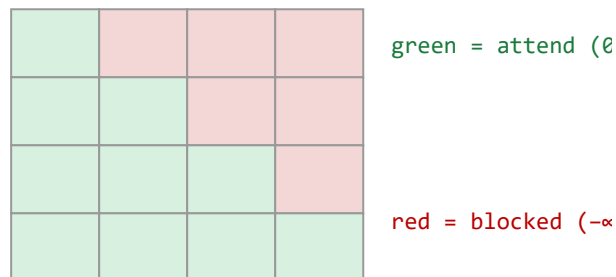


Figure 4. Lower-triangular causal mask: token L attends only to positions $\leq L$.

(b) Padding mask (encoder & decoder). Batches pad short sequences to equal length with `<pad>` tokens. The padding mask sets attention scores to $-\infty$ at pad positions so they contribute nothing and don't pollute the softmax.

(c) MLM token masking (BERT pre-training objective). A *different* notion: 15% of input tokens are selected; of those, **80%** are replaced with `[MASK]`, **10%** with a random token, **10%** are left unchanged. The model predicts the original tokens. The 10% random/unchanged tweak stops the model from only ever working when it sees `[MASK]` (which never appears at fine-tuning time).

Part 4 — Transformers: families, multi-head, positional encoding & parameter counts

4.1 The three architecture families (TA priority)

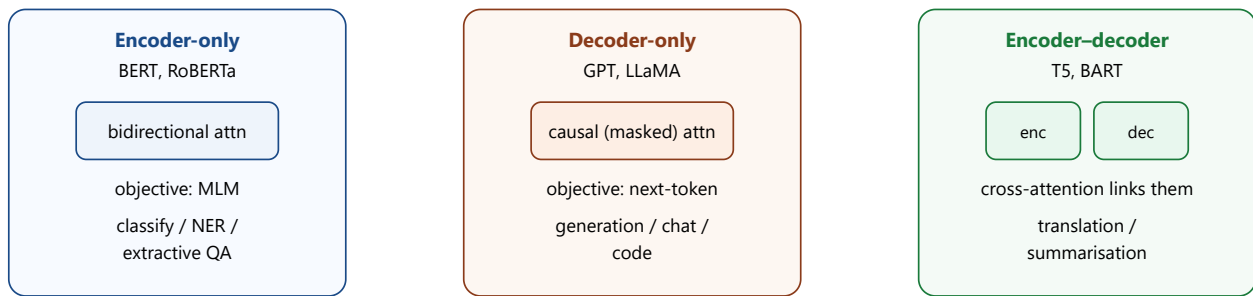


Figure 5. The three families. Translation can use decoder-only OR encoder-decoder — never encoder-only (it cannot generate).

Why decoder-only can drop positional encoding (a known MCQ). The causal mask itself makes position i see exactly i previous tokens — the *count* of attendable tokens encodes the position. An encoder is bidirectional and symmetric, so without PE it is genuinely permutation-invariant (a “bag of words”) and *cannot* recover order.

4.2 Multi-head attention

A single softmax head tends to be “peaky” — it focuses on one relationship. **Multi-head** runs h attention computations in parallel, each in its own d/h -dim subspace, so the model can attend to several relationships at once (e.g. subject *and* object):

```
head_i = Attention(XW_Q^i, XW_K^i, XW_V^i),    each projection maps d → d/h
MHA(X) = Concat(head_1, ..., head_h) · W_O
```

Crucially, the parameter count of MHA is the **same** as one full-width head: the per-head projections are $d \times (d/h)$, and h of them stack back up to $d \times d$.

4.3 Positional encoding and its parameter count (TA priority)

Two common schemes:

```
Sinusoidal (fixed):  PE(pos, 2i)   = sin( pos / 10000^{2i/d} )
                    PE(pos, 2i+1) = cos( pos / 10000^{2i/d} )
Learned absolute:    a lookup table of shape (max_len × d)
```

Parameter count — the exam point:

- **Sinusoidal PE has ZERO trainable parameters** (it is a fixed function of position).
- **Learned absolute PE has $\text{max_len} \times d$ parameters** (one vector per position).

Relative schemes (RoPE, ALiBi, T5 buckets) add few or zero extra parameters too.

4.4 🧩 Flagship numerical — full transformer parameter count

 **Worked numerical (≈8 marks).** Count the parameters of a transformer **encoder** with: $d = d_{\text{model}} = 512$, layers $L = 6$, heads $h = 8$, FFN hidden $d_{\text{ff}} = 2048$, vocab $V = 30000$, max length $n_{\text{pos}} = 512$, learned positional embeddings. Ignore biases and LayerNorm first, then add them. Output projection is **tied** with the input embedding.

Formulas to use (count each weight matrix as rows $\times$ columns).

Attention per layer : W_Q, W_K, W_V, W_O are each $d \times d \rightarrow 4 \cdot d^2$
 FFN per layer : W_1 is $d \times d_{\text{ff}}$, W_2 is $d_{\text{ff}} \times d \rightarrow 2 \cdot d \cdot d_{\text{ff}}$
 Per layer (weights) : $4d^2 + 2 \cdot d \cdot d_{\text{ff}}$
 All L layers : $L \cdot (4d^2 + 2 \cdot d \cdot d_{\text{ff}})$
 Token embedding : $V \cdot d$
 Positional (Learned) : $n_{\text{pos}} \cdot d$ (sinusoidal would be 0)
 Total = $L \cdot (4d^2 + 2 \cdot d \cdot d_{\text{ff}}) + V \cdot d + n_{\text{pos}} \cdot d$ (tied output head $\Rightarrow$ no extra)

Step 1 — per-layer attention. Four $d \times d$ matrices W_Q, W_K, W_V, W_O :

$$4 \cdot d^2 = 4 \cdot 512^2 = 4 \cdot 262,144 = 1,048,576$$

(Multi-head splits this into 8 heads but the total is unchanged.)

Step 2 — per-layer FFN. W_1 is $d \times d_{\text{ff}}$, W_2 is $d_{\text{ff}} \times d$:

$$2 \cdot d \cdot d_{\text{ff}} = 2 \cdot 512 \cdot 2048 = 2,097,152$$

Step 3 — per-layer total (weights only).

$$1,048,576 + 2,097,152 = 3,145,728 \approx 3.15\text{M per Layer}$$

Step 4 — all layers.

$$6 \times 3,145,728 = 18,874,368 \approx 18.9\text{M}$$

Step 5 — embeddings.

token embedding : $V \cdot d = 30000 \cdot 512 = 15,360,000$
 positional (Learned): $n_{\text{pos}} \cdot d = 512 \cdot 512 = 262,144$

(Sinusoidal PE here would contribute 0 instead of 262,144 — state this if asked.)

Step 6 — total (weights only, tied output head).

$$18,874,368 + 15,360,000 + 262,144 = 34,496,512 \approx 34.5\text{M parameters}$$

Step 7 — add biases and LayerNorm (the fine print). Per layer: attention biases $4d$, FFN biases $d_{\text{ff}} + d$, two LayerNorms $2 \cdot 2d$:

$$4 \cdot 512 + (2048 + 512) + 4 \cdot 512 = 2048 + 2560 + 2048 = 6,656 \text{ per Layer}$$

$$\times 6 \text{ layers} = 39,936 \quad (\approx 0.04\text{M} - \text{under } 0.2\% \text{ of the total})$$

So the grand total is $\approx 34.54M$. **Takeaway for the exam:** the dominant terms are the FFN ($8d^2$ per layer with $d_{ff}=4d$) and the token embedding ($V \cdot d$); biases and LayerNorm are negligible; if the output head is **untied**, add another $V \cdot d = 15.36M$.

Part 5 — Pre-training vs fine-tuning, and the fine-tuning trade-offs (TA priority)

5.1 Pre-training vs fine-tuning

	Pre-training	Fine-tuning
Data	massive, unlabelled corpus	small, task-specific labelled set
Objective	self-supervised (MLM or next-token)	supervised task loss
Cost	enormous (one-time)	cheap (per task)
Goal	learn general language representations	specialise to one task/domain

Why base models fail to follow instructions (a known MCQ). Not size, not “no RLHF” — the primary reason is that **most pre-training data is not in instruction→response format**, so a base model *continues* text rather than *answering* an instruction.

5.2 Last-layer vs full fine-tuning (performance + compute trade-off)

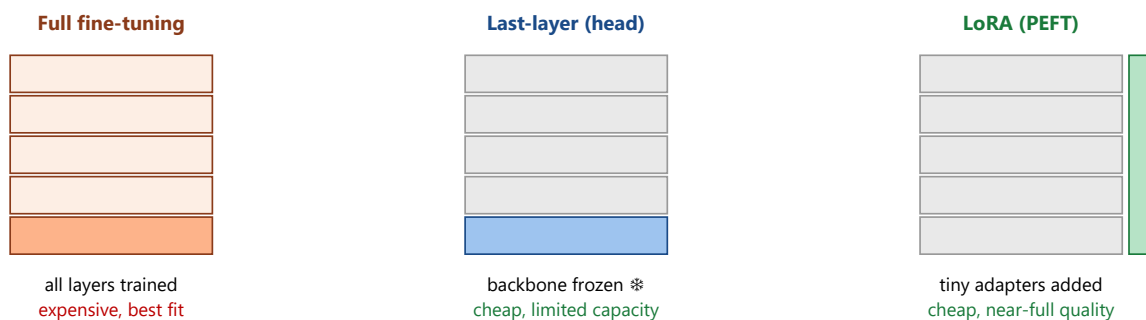


Figure 6. Grey = frozen, coloured = trained. Full FT updates everything; last-layer trains only the head; LoRA injects small trainable matrices while freezing the backbone.

The trade-off (be ready to argue both sides):

- **Full fine-tuning** updates *all* weights → highest accuracy and best at large domain shifts, but it stores gradients + optimizer state for every parameter ($\approx 3\text{--}4\times$ the model size in memory), is slow, and overfits on small datasets. Each task needs a *full copy* of the model.
- **Last-layer fine-tuning** freezes the backbone and trains only the task head → tiny memory/compute for the update, fast, robust on small data, but limited capacity: if the frozen features don't suit the new task (large domain shift), accuracy plateaus well below full FT.

 Worked numerical (≈6 marks). Trainable-parameter ratio.

A BERT-base encoder has $\approx 110\text{M}$ parameters and hidden size $d = 768$. You fine-tune it for **3-class** classification.

- How many parameters does last-layer (linear head) fine-tuning train?
- How many does full fine-tuning train? Give the ratio.
- One sentence on the memory implication.

Solution.

Formula to use. A linear classification head is one weight matrix $d \times C$ plus C biases, where $C =$ number of classes:

$$\begin{aligned}\text{head params} &= d \cdot C + C = (d + 1) \cdot C \\ \text{ratio} &= \text{trainable} / \text{total}\end{aligned}$$

Given. $d = 768$, $C = 3$, total model $\approx 110,000,000$.

Step 1 — (a) last-layer trainable params. Substitute $d = 768$, $C = 3$:

$$\text{head} = d \cdot C + C = 768 \cdot 3 + 3 = 2304 + 3 = 2,307 \text{ parameters}$$

Step 2 — (b) full FT and the ratio. Full FT trains every parameter:

$$\begin{aligned}\text{full FT} &= 110,000,000 \\ \text{ratio} &= 2,307 / 110,000,000 \approx 2.1 \times 10^{-5} \approx 0.002\%\end{aligned}$$

Step 3 — (c) memory. The optimizer (Adam) stores, per trainable parameter, the gradient + 2 moment estimates ≈ 3 values $\times 4$ bytes:

$$\begin{aligned}\text{full FT update state} &\approx 110\text{M} \times 3 \times 4 \text{ bytes} \approx 1.3 \text{ GB} \\ \text{Last-layer update state} &\approx 2,307 \times 3 \times 4 \text{ bytes} \approx 28 \text{ KB}\end{aligned}$$

Answer. (a) 2,307 (b) 110M, ratio $\approx 0.002\%$ (c) ~ 1.3 GB vs ~ 28 KB of optimizer state. **Interpretation.**

The forward pass costs about the same either way; the huge saving from last-layer FT is in the backward pass and optimizer state, which scale with the number of trainable parameters.

5.3 PEFT and LoRA (contrast + trade-offs)

Method	Trainable share	Idea
Full FT	100%	update everything (baseline)
Adapters	1–3%	small bottleneck MLP inserted per block
LoRA	0.1–1%	freeze W , learn low-rank update $\Delta W = BA$, $r \ll d$; merge at inference (no latency)
QLoRA	LoRA + 4-bit base	fits very large models on one GPU
Prompt tuning	<0.1%	learn a soft prompt at the input only
Prefix tuning	<1%	learn soft prompts at every layer's K, V

Worked numerical (≈5 marks). LoRA savings.

Apply LoRA with rank $r = 8$ to a single weight matrix $W \in \mathbb{R}^{768 \times 768}$.

- a. Trainable params with LoRA vs full. (b) Percentage. (c) If applied to W_Q and W_V in all 12 layers of BERT-base, total LoRA params?

Solution.

Formula to use. LoRA freezes W and learns a low-rank update $\Delta W = B \cdot A$, where B is $d \times r$ and A is $r \times d$ (so ΔW is still $d \times d$ but built from far fewer numbers):

$$\begin{aligned} \text{LoRA params (per matrix)} &= (d \cdot r) + (r \cdot d) = 2 \cdot d \cdot r \\ \text{Full update (per matrix)} &= d \cdot d = d^2 \end{aligned}$$

Given. $d = 768$, $r = 8$.

Step 1 — (a) per-matrix counts.

$$\begin{aligned} \text{LoRA} &= 2 \cdot d \cdot r = 2 \cdot 768 \cdot 8 = 12,288 \\ \text{Full} &= d^2 = 768^2 = 589,824 \end{aligned}$$

Step 2 — (b) percentage of full.

$$12,288 / 589,824 = 0.0208 = 2.08\%$$

Step 3 — (c) across the model. Apply to 2 matrices (W_Q , W_V) in each of 12 layers:

$$\text{total} = 12,288 \times 2 \text{ matrices} \times 12 \text{ layers} = 294,912 \approx 0.29M \text{ trainable}$$

Answer. (a) 12,288 vs 589,824 (b) 2.08% (c) ≈0.29M. **Interpretation.** That 0.29M is ≈0.27% of BERT-base's 110M, yet LoRA usually recovers most of full-FT accuracy. **Trade-off:** far less memory, and you keep one frozen base and swap a tiny adapter per task — at a small risk of underfitting very large domain shifts, where full FT still wins.

Part 6 — Prompting (intuition-level, as the TA described)

Zero-shot prompting: instruction only, no examples — relies entirely on what the model learned in pre-training/instruction-tuning. “*Translate to French: ‘good morning’*”

Few-shot prompting / in-context learning (ICL): prepend a handful of input→output examples in the prompt. The model infers the pattern and applies it — **no weights are updated**; “learning” happens purely in the forward pass via the context.

```
EN: hello -> FR: bonjour
EN: thanks -> FR: merci
EN: good morning -> FR: ?      ← model completes
```

Chain-of-thought (CoT): include intermediate reasoning steps in the examples (or simply append “Let’s think step by step” for *zero-shot CoT*). Helps on multi-step arithmetic/logic by giving the model room to compute before answering.

Self-consistency: sample several CoT paths and take the majority answer.

Prompt engineering vs prompt tuning: prompt *engineering* is manual wording of discrete text; prompt *tuning* learns continuous “soft prompt” vectors via gradient descent (a PEFT method) — different things despite the similar name.

Part 7 — RLHF & Alignment (the long-answer / derivation section)

This is where the TA expects pipelines and derivations. Master the three-stage pipeline, the Bradley-Terry derivation, the RLHF objective, and PPO-CLIP.

7.1 LLM as an RL policy

RL term	Meaning for an LLM
State s_t	prompt + all tokens generated so far
Action a_t	the next token emitted
Policy $\pi_\theta(a \setminus s)$	the LLM itself (probability over vocab)
Environment	the abstract (input, output, reward) loop — <i>not</i> a game simulator
Reward $r(x, y)$	reward-model score of the full response (sparse, at the end)
Return G_t	$\sum_k \gamma^k r_{\{t+k+1\}}$
Value / Q / Advantage	$V(s)=E[G \setminus s]$; $Q(s, a)=E[G \setminus s, a]$; $A(s, a)=Q-V$ (how much better an action is than average)

7.2 The RLHF pipeline (three stages)

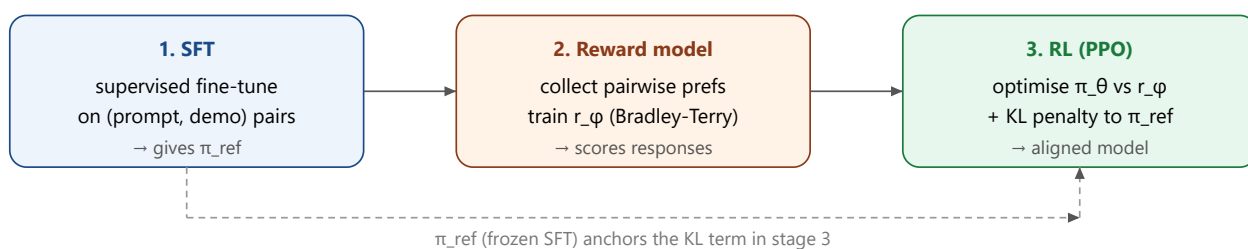


Figure 7. RLHF: SFT → reward model → PPO. The frozen SFT model serves as the reference policy π_{ref} for the KL penalty.

7.3 🎲 Bradley-Terry reward-model derivation (TA flagged this explicitly)

 Derivation (≈6–8 marks). Derive the reward-model loss from the Bradley-Terry model.

Setup. The Bradley-Terry model assigns each item i a latent “strength” s_i and models the probability that i beats j as proportional to its strength:

$$P(i > j) = \exp(s_i) / (\exp(s_i) + \exp(s_j))$$

Step 1 — reduce to a sigmoid. Divide numerator and denominator by $\exp(s_i)$:

$$P(i > j) = 1 / (1 + \exp(s_j - s_i)) = \sigma(s_i - s_j)$$

where $\sigma(z) = 1/(1+e^{-z})$ is the logistic sigmoid. So a preference depends only on the **difference** of strengths.

Step 2 — let the reward model supply the strengths. For a prompt x with a preferred (winning) response y_w and a dispreferred y_L , define the strength of a response to be its reward $r_\phi(x, y)$. Then:

$$P(y_w > y_L \mid x) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_L))$$

Step 3 — maximum likelihood over the preference dataset. With dataset $D = \{(x, y_w, y_L)\}$, the log-likelihood is $\sum \log \sigma(r_\phi(x, y_w) - r_\phi(x, y_L))$. Training **minimises the negative** of this:

$$L(\phi) = - E_{\{(x, y_w, y_L) \sim D\}} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_L))]$$

Why this loss makes sense. It pushes $r_\phi(x, y_w)$ above $r_\phi(x, y_L)$ by as large a margin as the data supports (sigmoid saturates, so it stops once the order is confident). The reward model is usually the SFT model with a scalar head replacing the vocabulary head. **Common MCQ:** the objective is to **maximise $\log \sigma(r_w - r_L)$** — not MSE, not KL.

 **Worked numerical (≈4 marks). One preference pair's loss.** Suppose $r_{\phi}(x, y_w) = 2.0$ and $r_{\phi}(x, y_l) = 0.5$. Compute the per-example loss.

Formula to use. From Step 3 above, the loss for one pair is

$$L = -\log \sigma(r_w - r_l), \quad \text{where } \sigma(z) = 1 / (1 + e^{-z})$$

r_w = reward of the preferred response, r_l = reward of the dispreferred one.

Step 1 — margin (the score gap the model gives).

$$z = r_w - r_l = 2.0 - 0.5 = 1.5$$

Step 2 — sigmoid of the margin (= predicted $P(y_w > y_l)$).

$$\sigma(1.5) = 1 / (1 + e^{-1.5}) = 1 / (1 + 0.2231) = 1 / 1.2231 = 0.8176$$

Step 3 — negative log.

$$L = -\log(0.8176) = 0.2014$$

Answer. $L \approx 0.20$. **Interpretation / check.** The model already ranks them correctly (margin +1.5), so the loss is small. If it had scored them equal ($z = 0$): $\sigma(0) = 0.5$, $L = -\log 0.5 = 0.693$ — a larger loss, pushing the rewards apart. As the margin grows, $\sigma \rightarrow 1$ and $L \rightarrow 0$.

7.4 The regularized RLHF objective and its optimal policy

 **Derivation (≈6–8 marks). State the RLHF objective and derive the optimal policy.**

Objective (KL-regularized reward maximization):

$$\max_{\theta} \mathbb{E}_{\{x, y \sim \pi_{\theta}(\cdot|x)\}} [r_{\phi}(x, y)] - \beta \cdot \text{KL}(\pi_{\theta}(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x))$$

Expanding the KL = $\mathbb{E}_{\{\pi_{\theta}\}} [\text{Log}(\pi_{\theta}/\pi_{\text{ref}})]$ folds it into the expectation:

$$\max_{\theta} \mathbb{E}_{\{y \sim \pi_{\theta}\}} [r_{\phi}(x, y) - \beta \text{Log}(\pi_{\theta}(y|x) / \pi_{\text{ref}}(y|x))]$$

Role of each piece. r_{ϕ} pulls the policy toward high-reward responses; the KL term (weight β) keeps it close to the frozen SFT reference π_{ref} . This prevents **reward hacking** (drifting into gibberish that fools the reward model) and preserves output diversity.

Derive the closed-form optimum. Treat $\pi = \pi_{\theta}(\cdot|x)$ as the variable and maximise pointwise subject to $\sum_y \pi(y) = 1$. The Lagrangian per response is

$$\pi(y) [r(y) - \beta \text{Log}(\pi(y)/\pi_{\text{ref}}(y))] + \lambda (\sum_y \pi(y) - 1)$$

Differentiate w.r.t. $\pi(y)$ and set to 0:

$$\begin{aligned} r(y) - \beta (\text{Log}(\pi(y)/\pi_{\text{ref}}(y)) + 1) + \lambda &= 0 \\ \Rightarrow \text{Log}(\pi(y)/\pi_{\text{ref}}(y)) &= r(y)/\beta + (\lambda/\beta - 1) \\ \Rightarrow \pi^*(y|x) &= (1/Z(x)) \cdot \pi_{\text{ref}}(y|x) \cdot \exp(r(y)/\beta) \end{aligned}$$

where $Z(x) = \sum_y \pi_{\text{ref}}(y|x) \exp(r(y)/\beta)$ normalises. **Reading:** the optimal aligned policy is the reference policy reweighted by the exponentiated reward — high-reward responses get up-weighted, but staying tethered to π_{ref} . (This identity is exactly what DPO exploits to skip the reward model and RL loop.)

7.5 Policy gradients, importance sampling, REINFORCE → PPO

REINFORCE (with baseline). The policy-gradient estimator, with a baseline $b(s) = \hat{V}(s)$ to cut variance:

$$\nabla_{\theta} J \approx \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \cdot (G_t - b(s_t))$$

Importance sampling. To reuse data sampled from an old policy π_{old} while optimising π_{θ} :

$$\mathbb{E}_{\{a \sim \pi_{\theta}\}} [f(a)] = \mathbb{E}_{\{a \sim \pi_{\text{old}}\}} [(\pi_{\theta}(a)/\pi_{\text{old}}(a)) \cdot f(a)], \quad \text{ratio } \rho_t = \pi_{\theta}(a_t|s_t) / \pi_{\text{old}}(a_t|s_t)$$

This lets PPO take several gradient steps on one batch of rollouts.

PPO-CLIP. Clip the ratio so a single update can't move the policy too far (a pessimistic lower bound on the objective):

$$L^{\text{CLIP}} = \mathbb{E}_t [\min(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) \cdot \hat{A}_t)] \quad (\epsilon \approx 0.1-0.2)$$

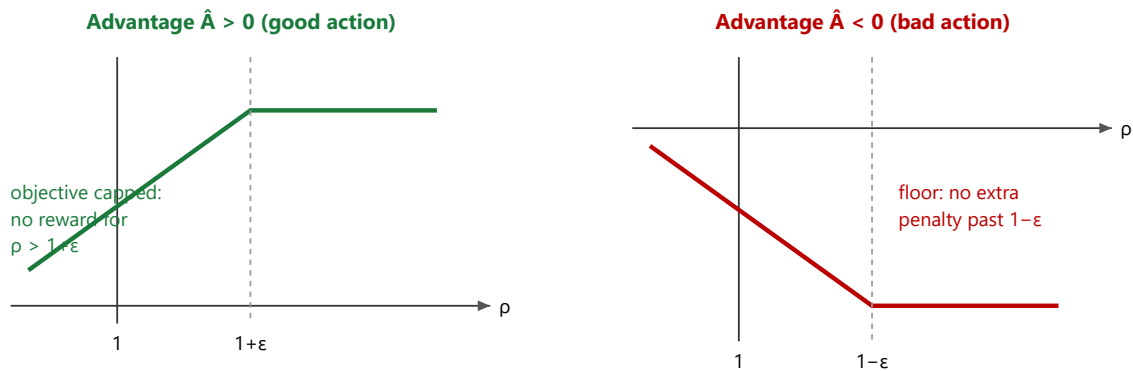


Figure 8. PPO clipping. For a good action ($\hat{A} > 0$) the gain is capped once p exceeds $1 + \epsilon$; for a bad action ($\hat{A} < 0$) the penalty is floored at $1 - \epsilon$. Either way the incentive to move the policy far in one step is removed.



Worked numerical (≈ 5 marks). PPO-CLIP objective. Let $\epsilon = 0.2$. Compute L^{CLIP} for: (a) $\hat{A} = +2$, $\rho = 1.5$. (b) $\hat{A} = +2$, $\rho = 0.7$. (c) $\hat{A} = -2$, $\rho = 1.5$.

Formula to use.

$$L^{\text{CLIP}} = \min(\rho \cdot \hat{A}, \text{clip}(\rho, 1 - \epsilon, 1 + \epsilon) \cdot \hat{A})$$

$\rho = \pi_{\theta} / \pi_{\text{old}}$ is the probability ratio, $\hat{A}$ the advantage, and $\text{clip}(\rho, \text{lo}, \text{hi})$ forces ρ back into $[\text{lo}, \text{hi}]$. With $\epsilon = 0.2$ the clip range is $[1 - 0.2, 1 + 0.2] = [0.8, 1.2]$. Evaluate the two terms and take the **minimum**.

(a) $\hat{A} = +2$, $\rho = 1.5$.

$$\begin{aligned} \text{term1} &= \rho \cdot \hat{A} &&= 1.5 \cdot 2 = 3.0 \\ \text{clip}(1.5 \rightarrow [0.8, 1.2]) &= 1.2 && \text{(1.5 is above 1.2, so pulled down)} \\ \text{term2} &= \text{clip} \cdot \hat{A} &&= 1.2 \cdot 2 = 2.4 \\ L^{\text{CLIP}} &= \min(3.0, 2.4) &&= 2.4 \end{aligned}$$

The clip **caps** the gain — no extra credit for moving ρ past 1.2.

(b) $\hat{A} = +2$, $\rho = 0.7$.

$$\begin{aligned} \text{term1} &= 0.7 \cdot 2 = 1.4 \\ \text{clip}(0.7 \rightarrow [0.8, 1.2]) &= 0.8; \quad \text{term2} = 0.8 \cdot 2 = 1.6 \\ L^{\text{CLIP}} &= \min(1.4, 1.6) = 1.4 \end{aligned}$$

Here the unclipped term is already smaller, so clipping has no effect.

(c) $\hat{A} = -2$, $\rho = 1.5$.

$$\begin{aligned} \text{term1} &= 1.5 \cdot (-2) = -3.0 \\ \text{clip}(1.5 \rightarrow [0.8, 1.2]) &= 1.2; \quad \text{term2} = 1.2 \cdot (-2) = -2.4 \\ L^{\text{CLIP}} &= \min(-3.0, -2.4) = -3.0 \end{aligned}$$

For a **bad** action ($\hat{A} < 0$) that the new policy made more likely, the $\min$ keeps the more-negative -3.0 , so the policy is **not** shielded from the penalty. **Answer.** (a) 2.4 (b) 1.4 (c) -3.0 . **Interpretation.** The asymmetry is the whole point: clipping removes the incentive to over-shoot on good actions, but still penalises over-shooting on bad ones — keeping each update conservative.

7.6 Constitutional AI (alignment without (much) human labelling)

- **SL-CAI**: the model critiques its own answer against a written “constitution”, revises it, and is fine-tuned on the revisions.
- **RL-CAI**: an AI judge (not humans) produces the pairwise preferences that train the reward model, which then drives PPO. This is “RLAIF” — RL from AI feedback.

Part 8 — Word embeddings: word2vec & GloVe

Intuition (distributional hypothesis). “You shall know a word by the company it keeps.” Words that appear in similar contexts should get similar vectors. Embeddings turn discrete tokens into dense vectors where geometry encodes meaning (king - man + woman ≈ queen).

word2vec comes in two flavours:

- **Skip-gram**: given the centre word, predict each surrounding context word.
- **CBOW** (continuous bag-of-words): given the (averaged) context words, predict the centre word.

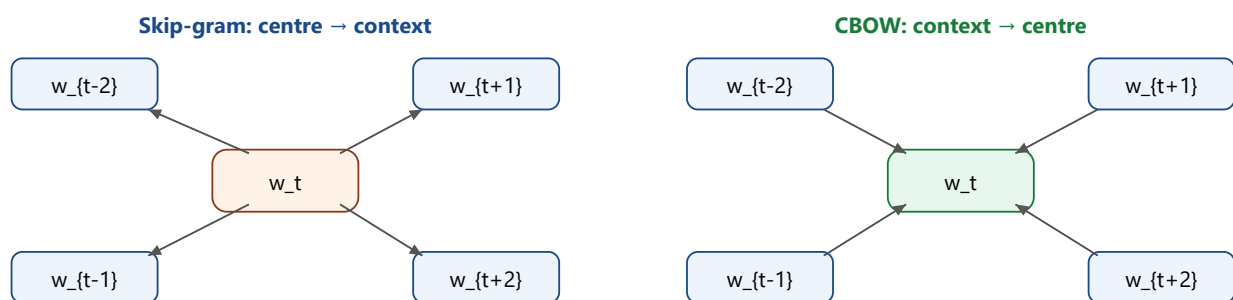


Figure 9. Skip-gram predicts each context word from the centre; CBOW predicts the centre from the averaged context.

Negative sampling (the efficient training trick). A full softmax over the vocabulary is too expensive, so skip-gram instead frames each (centre c , context o) pair as binary classification: push the true pair together and a few k sampled “noise” words apart:

$$L = -\log \sigma(u_o \cdot v_c) - \sum_{n \in \text{Neg}} \log \sigma(-u_n \cdot v_c)$$

Noise words are drawn from the unigram distribution raised to the 3/4 power, $P(w) \propto \text{count}(w)^{3/4}$, which boosts rare words relative to plain frequency.

GloVe takes a different route: it fits embeddings directly to the *global* co-occurrence counts X_{ij} (how often word j appears in the context of word i):

$$J = \sum_{\{i,j\}} f(X_{ij}) \cdot (w_i \cdot \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2$$

$f(X_{ij})$ is a weighting that damps very frequent pairs. **Exam trap:** GloVe uses **no** negative sampling (it regresses on log-counts) — the opposite of word2vec.

 **Worked numerical (≈5 marks). Skip-gram negative-sampling loss.** For one centre/context pair, the model gives dot products $u_o \cdot v_c = 2.0$ (true context) and two negatives $u_{n1} \cdot v_c = 0.5$, $u_{n2} \cdot v_c = -1.0$. Compute the loss.

Formula to use.

$$L = -\log \sigma(u_o \cdot v_c) - \sum_{n} \log \sigma(-u_n \cdot v_c)$$

The first term wants the true pair's dot product high (so $\sigma \rightarrow 1$); each negative term wants $-u_n \cdot v_c$ high, i.e. the noise dot product low/negative. Use $\sigma(z) = 1/(1+e^{-z})$.

Step 1 — true (positive) term. Plug in $u_o \cdot v_c = 2.0$:

$$\begin{aligned}\sigma(2.0) &= 1/(1+e^{-2.0}) = 1/(1+0.1353) = 0.8808 \\ \text{term_true} &= -\log(0.8808) = 0.1269\end{aligned}$$

Step 2 — negative term 1. Note the sign flip: the argument is $-u_{n1} \cdot v_c = -0.5$:

$$\begin{aligned}\sigma(-0.5) &= 1/(1+e^{0.5}) = 1/(1+1.6487) = 0.3775 \\ \text{term_n1} &= -\log(0.3775) = 0.9741\end{aligned}$$

Step 3 — negative term 2. Argument $-u_{n2} \cdot v_c = -(-1.0) = +1.0$:

$$\begin{aligned}\sigma(1.0) &= 1/(1+e^{-1.0}) = 1/(1+0.3679) = 0.7311 \\ \text{term_n2} &= -\log(0.7311) = 0.3133\end{aligned}$$

Step 4 — sum.

$$L = 0.1269 + 0.9741 + 0.3133 = 1.414$$

Answer. $L \approx 1.41$. **Interpretation.** Negative $n1$ contributes the most (0.97): its dot product was $+0.5$, meaning the model wrongly placed that noise word near the centre — so the gradient pushes them apart hardest. Negative $n2$ (dot -1.0 , already far) contributes little.

Part 9 — Smoothing techniques (full family)

The problem. MLE n-grams give probability 0 to any sequence containing an unseen n-gram, making perplexity infinite. Smoothing reserves probability mass for unseen events. (Add-k from Part 1.3 is the simplest.)

Method	Formula / idea	Key property
Add-k (Laplace)	$(c + k) / (N + kV)$	simple; over-smooths for large V
Good-Turing	reweight using counts-of-counts: $c^* = (c+1) \cdot N_{\{c+1\}} / N_c$	estimates unseen mass as N_1/N
Jelinek-Mercer (interpolation)	$\lambda \cdot P_n + (1-\lambda) \cdot P_{\{n-1\}}$ (recurse to unigram)	always mixes lower orders
Katz backoff	use the n -gram if seen (discounted), else <i>back off</i> to $(n-1)$ -gram	discount + redistribute
Kneser-Ney	absolute discount + continuation probability : how many <i>distinct</i> contexts a word follows	best classical smoother

Why Kneser-Ney's continuation idea matters. "Francisco" is frequent but only ever follows "San". A plain unigram would give "Francisco" high backoff probability; Kneser-Ney instead asks "*in how many distinct contexts does this word appear?*" — "Francisco" scores low, "the" scores high — which is what a backoff distribution should reward.

 **Worked numerical (≈5 marks). Good-Turing.** A corpus has $N = 100$ tokens. The counts-of-counts are $N_1 = 10$ (words seen once), $N_2 = 4$ (seen twice), $N_3 = 2$. (a) Good-Turing adjusted count c^* for a word seen once. (b) Total probability mass assigned to **unseen** events.

Solution.

Formula to use.

$$\begin{aligned} \text{Adjusted count:} \quad c^* &= (c+1) \cdot N_{\{c+1\}} / N_c \\ \text{Unseen mass:} \quad P(\text{unseen}) &= N_1 / N \end{aligned}$$

N_c = the number of distinct words that occur exactly c times (the "count of counts"), and N = total tokens. The idea: estimate how often a c -count word should occur using how many $(c+1)$ -count words exist.

Given. $N = 100$, $N_1 = 10$, $N_2 = 4$, $N_3 = 2$.

Step 1 — (a) adjusted count for $c = 1$. Substitute $c = 1$ (so we need $N_{\{c+1\}} = N_2$):

$$c^* = (1+1) \cdot N_2 / N_1 = 2 \cdot 4 / 10 = 8/10 = 0.8$$

Step 2 — (b) unseen mass.

$$P(\text{unseen}) = N_1 / N = 10 / 100 = 0.10$$

Answer. (a) $c^* = 0.8$ (b) 10% of the probability mass goes to unseen events. **Interpretation.** A once-seen word is discounted to behave as if seen only 0.8 times; the freed-up 0.10 of mass is handed to events never observed — exactly what MLE (which gives them 0) cannot do.

Part 10 — Pre-training objectives & the model families in depth

The four headline models differ mainly in their **pre-training objective**, which in turn dictates which family they belong to.

MLM (BERT, encoder)

in : the [MASK] sat on the [MASK]
out: predict {cat, mat} at masked slots (bidirectional)

Causal LM (GPT, decoder)

in : the cat sat on the
out: predict next token “mat” (left-to-right only)

Span corruption (T5, enc-dec)

in : the <X> sat on <Y> mat
out: <X> cat <Y> the (sentinels mark dropped spans)

Denoising (BART, enc-dec)

corrupt input by: token mask / delete /
infill spans / sentence permute / doc rotate
out: reconstruct the original clean text

BERT enc-only → understanding (classify/NER/QA)

GPT dec-only → generation / chat / code

T5 enc-dec → any text-to-text task

BART enc-dec → summarisation / generation

Figure 10. The same sentence under four pre-training objectives; the objective determines the family and what the model is best at.

- **BERT (MLM + NSP).** Masks 15% of tokens (80/10/10 rule, Part 3.2) and predicts them using *bidirectional* context; the original also added Next-Sentence Prediction (RoBERTa dropped NSP and improved).
- **GPT (causal LM).** Predicts the next token left-to-right; this is exactly an autoregressive language model and supports generation.
- **T5 (span corruption).** Replaces contiguous spans with single sentinel tokens; the decoder emits the missing spans. Everything (translation, classification, summarisation) is cast as text-to-text.
- **BART (denoising autoencoder).** Corrupts text with several noises (masking, deletion, infilling, sentence permutation, document rotation) and reconstructs the original — strong for summarisation.

MLM vs autoregressive, in one line: MLM sees the whole sentence (bidirectional, great for *understanding*) but isn't a natural generator; autoregressive sees only the left context (great for *generation*).

 **Worked numerical (≈3 marks). MLM masking budget.** A 200-token sequence is pre-trained with BERT-style masking. How many tokens become [MASK], random, and unchanged?

Formula to use (BERT's 15% rule, split 80/10/10).

selected = 15% of all tokens
of those: 80% → [MASK], 10% → random token, 10% → unchanged

Step 1 — how many tokens are selected for prediction.

selected = $0.15 \times 200 = 30$ tokens

Step 2 — split those 30 by the 80/10/10 rule.

[MASK] = $0.80 \times 30 = 24$
random = $0.10 \times 30 = 3$
unchanged = $0.10 \times 30 = 3$ (check: $24 + 3 + 3 = 30$ ✓)

Answer. 24 [MASK], 3 random, 3 unchanged. **Interpretation.** The 10% random + 10% unchanged keep the model from assuming the answer only ever sits under a literal [MASK] token (which never appears during fine-tuning).

Part 11 — Instruction tuning & synthetic instruction data

Instruction tuning (SFT) fine-tunes a base model on many (instruction, response) pairs so it learns to *answer* rather than *continue*. The bottleneck is getting enough high-quality instruction data — hence synthetic-data methods:

Method	Distinct trick
Self-Instruct	Bootstrap: start from ~175 seed tasks, have the LM generate new instructions + instances, filter for quality/diversity, add back to the pool.
Evol-Instruct (WizardLM)	An Instruction Evolver rewrites prompts to be harder — <i>in-depth</i> (add constraints, deepen, concretize, increase reasoning steps, complicate input) and <i>in-breadth</i> (invent new instructions).
Orca	Explanation tuning: train on GPT-4's step-by-step <i>reasoning traces</i> , not just final answers, so the student learns the process.
Instruction Back-translation (Humpback)	Take raw web text as a <i>response</i> and have an LM invent the <i>instruction</i> it answers; self-curate the best pairs.

Why SFT alone is not enough (→ RLHF). SFT teaches the *form* of an answer but not *which* of many valid answers is most helpful/harmless/honest. Classic failure: “best way to lose weight quickly?” → an SFT model may say “stop eating entirely.” Alignment (Part 7) fixes the *preference*, not the format.

Part 12 — Decoding strategies

Once trained, how do we turn the next-token distribution into text?

Strategy	What it does	When
Greedy	take argmax each step	deterministic; can be repetitive and <i>locally</i> optimal only
Beam search (width B)	keep the B highest-probability partial sequences	translation/summarisation; larger B → higher-probability sequences (a trap: not <i>smaller</i>)
Pure sampling	sample from the full distribution	creative but can go off the rails
Temperature	$\text{softmax}(z/T)$: $T < 1$ sharper, $T > 1$ flatter, $T \rightarrow 0 \approx$ greedy	controls randomness
Top-k	sample only from the k most likely tokens	caps the tail
Top-p (nucleus)	sample from the smallest set whose cumulative prob $\geq p$	adaptive tail cut

Why greedy is not optimal. Greedy commits to the best *first* token, which can foreclose a globally better sequence. Beam search hedges by carrying several hypotheses.

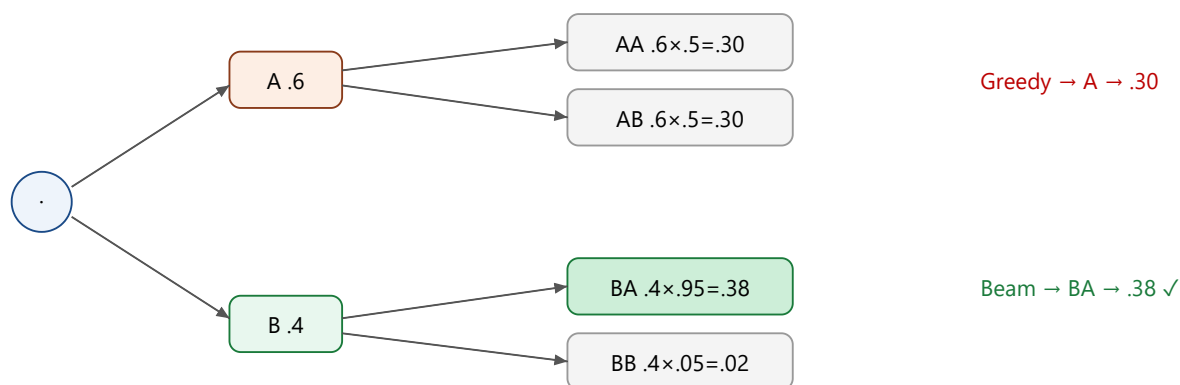


Figure 11. Greedy commits to A (0.6) and lands on 0.30; beam search keeps B alive and finds the better BA path (0.38).

 **Worked numerical (≈5 marks). Greedy vs beam.** Vocabulary $\{A, B\}$, two steps. Step 1: $P(A)=0.6$, $P(B)=0.4$. Continuations: from A, $P(A)=0.5$, $P(B)=0.5$; from B, $P(A)=0.95$, $P(B)=0.05$. (a) What does greedy output and with what probability? (b) What does beam search (width 2) output? (c) Comment.

Solution.

Method. A sequence's probability is the **product** of its per-step probabilities: $P(t_1 t_2) = P(t_1) \times P(t_2 | t_1)$. Greedy takes the single most-likely token at each step (never looking back). Beam search (width B) keeps the B best partial sequences at every step and expands all of them.

Step 1 — (a) greedy. Step 1 most-likely token: A ($0.6 > 0.4$). From A, most-likely next: tie 0.5, pick one:

$$P(\text{greedy path}) = P(A) \times \text{best continuation} = 0.6 \times 0.5 = 0.30$$

Step 2 — (b) beam, width 2. Enumerate every length-2 path and its product:

$$\begin{aligned} AA &= P(A) \cdot P(A|A) = 0.6 \times 0.5 = 0.30 \\ AB &= P(A) \cdot P(B|A) = 0.6 \times 0.5 = 0.30 \\ BA &= P(B) \cdot P(A|B) = 0.4 \times 0.95 = 0.38 \quad \leftarrow \text{highest} \\ BB &= P(B) \cdot P(B|B) = 0.4 \times 0.05 = 0.02 \end{aligned}$$

Beam keeps the best → **BA with 0.38. Answer.** (a) greedy → A., 0.30. (b) beam-2 → BA, 0.38. (c) **Why they differ.** Greedy committed to the high first token A, but B — though lower at step 1 — leads to a much sharper continuation (0.95), giving a higher overall product. Greedy's locally-best choice was globally suboptimal; this is exactly why larger beams tend to find higher-probability sequences.

Part 13 — Retrieval-Augmented Generation (RAG)

Why. Parametric LLMs hallucinate and go stale. RAG fetches relevant passages from an external corpus at query time and conditions generation on them → more factual, updatable, and *inspectable* (you can see which passage caused an error and fix the retriever).

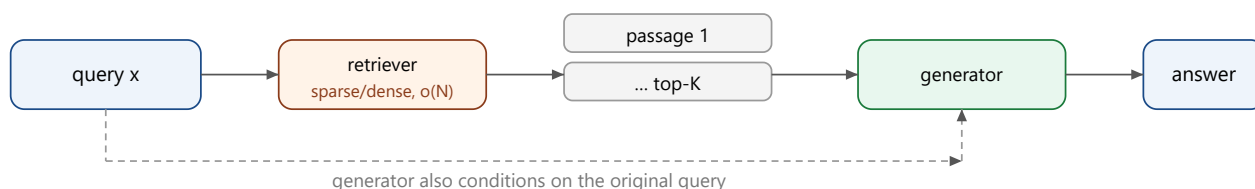


Figure 12. RAG: the retriever returns top-K passages (in sub-linear $o(N)$ time via an index); the generator conditions on query + passages.

Model	What's retrieved	How used	Evidence per output
REALM	single latent passage	masked-LM pre-training with retrieval	single doc
RAG-Sequence	top-K passages	each passage generates the whole output; marginalise over passages	single doc per output
RAG-Token	top-K passages	marginalise per <i>token</i>	multiple docs
FiD (Fusion-in-Decoder)	top-K passages	encode each separately; decoder cross-attends over <i>all</i>	multiple docs

Exam points: single-document evidence $\Rightarrow$ **REALM and RAG-Sequence**; multi-document fusion $\Rightarrow$ **RAG-Token and FiD**. Index lookup is $\mathbf{o(N)}$ (sub-linear), not $\Theta(N)$. For complex questions, **multi-hop** retrieval extracts a sub-question from the first results to drive a second retrieval.

Part 14 — Agentic frameworks

Non-agentic (zero-shot) vs agentic: a non-agentic workflow produces the whole answer in one pass; an agentic one *loops* — plan, act, observe, revise — until satisfied (e.g. essay: outline $\rightarrow$ research $\rightarrow$ draft $\rightarrow$ revise).

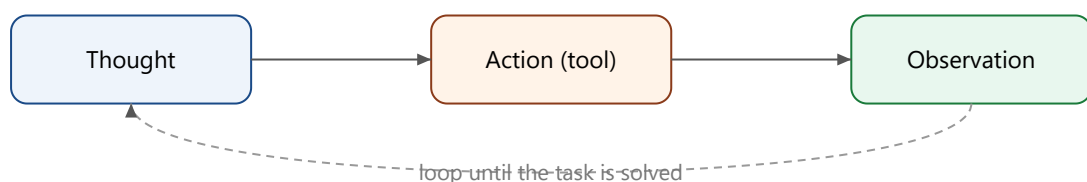


Figure 13. ReAct interleaves reasoning (*Thought*) with tool calls (*Action*) and their results (*Observation*), looping until done.

Framework	Key idea
CoT	reasoning in text only, no tools
ReAct	Thought → Action → Observation cycle; the observation injects external/tool info, unlike pure CoT
ReWOO	decouples planning from observation: plan all steps once (with placeholders) → Worker runs tools → Solver fuses → fewer tokens than ReAct
Reflexion	verbal reinforcement: keeps a self-reflection memory across trials to improve — no weight updates
Self-Refine	Initial → Feedback → Refine within a single trial; the Feedback stage critiques the draft
HuggingGPT	controller LLM: Task Planning → Model Selection → Task Execution → Response Generation (Task Planning parses the request into sub-tasks)

Part 15 — DPO and GAE (rounding out alignment)

Direct Preference Optimization (DPO). Recall the optimal RLHF policy (Part 7.4): $\pi^*(y|x) \propto \pi_{\text{ref}}(y|x) \exp(r(x,y)/\beta)$. Inverting this expresses the reward in terms of the policy, so the Bradley-Terry preference loss can be written *directly* in policy probabilities — **no reward model, no RL loop**:

$$L_{\text{DPO}} = -\log \sigma \left(\beta \left[\log \left(\frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} \right) - \log \left(\frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right] \right)$$

It simply raises the (reference-relative) log-prob of the preferred response above the dispreferred one. Simpler and more stable than PPO; PPO can still edge it out at scale with a good reward model.

Generalized Advantage Estimation (GAE). PPO needs an advantage estimate $\hat{A}_t$. GAE trades bias for variance with a λ -weighted sum of TD residuals:

$$\begin{aligned} \delta_t &= r_t + \gamma \hat{V}(s_{t+1}) - \hat{V}(s_t) && \text{(one-step TD error)} \\ \hat{A}_t &= \sum_{l \geq 0} (\gamma \lambda)^l \cdot \delta_{t+l} && (\lambda=0 \rightarrow \text{low-variance TD}; \lambda=1 \rightarrow \text{high-variance MC}) \end{aligned}$$

Part 16 — Long-answer drill set (attempt before peeking)

Full solutions are above in the relevant sections; targets are the difficulty the TA described.

- (8)** Derive the transformer encoder parameter count for $d=768$, $L=12$, $d_{\text{ff}}=3072$, $V=30522$, $n_{\text{pos}}=512$ (BERT-base config), output head tied. (Method: Part 4.4. Expect $\approx 110M$.)

2. **(6)** A test set of 100 tokens gets total $\log_2$ -probability -650 under model A and -700 under model B. Which model is better, and what are the two perplexities? ($H_A=6.5 \rightarrow PPL=90.5$; $H_B=7.0 \rightarrow PPL=128$; A is better.)
3. **(7)** Derive the Bradley-Terry reward loss from first principles and explain why the KL term is needed in the downstream RLHF objective. (Parts 7.3–7.4.)
4. **(6)** For $\epsilon=0.1$, tabulate L^{CLIP} for $\hat{A}=+1$ at $p \in \{0.85, 1.0, 1.2\}$ and for $\hat{A}=-1$ at the same ratios. (Apply Part 7.5.)
5. **(5)** Compare last-layer, full, and LoRA fine-tuning of a 350M-param model with $d=1024$ for a 5-class task: trainable parameters of each (LoRA $r=16$ on W_Q, W_V across 24 layers). (Method: Parts 5.2–5.3.)
6. **(6)** Compute one head of self-attention for 2 tokens with $Q = \begin{bmatrix} 1, 1 \\ 0, 2 \end{bmatrix}$, $K = \begin{bmatrix} 1, 0 \\ 1, 1 \end{bmatrix}$, $V = \begin{bmatrix} 2, 0 \\ 0, 2 \end{bmatrix}$, $d_k=2$; report both output rows. (Method: Part 3.1.)
7. **(5)** Compute the skip-gram negative-sampling loss for $u_o \cdot v_c = 1.5$ with three negatives of dot product $1.0, 0.0, -2.0$. Which negative hurts most and why? (Method: Part 8.)
8. **(5)** A corpus has $N = 200$, $N_1 = 30$, $N_2 = 12$. Give the Good-Turing adjusted count for a once-seen word and the total unseen-event mass. (Method: Part 9.)
9. **(6)** Name the pre-training objective of BERT, GPT, T5 and BART, and for each state the architecture family and one task it excels at. (Method: Part 10.)
10. **(5)** Two-step decoding, vocab $\{A, B\}$: step 1 $P(A)=0.7, P(B)=0.3$; from A $P(A)=0.4, P(B)=0.6$; from B $P(A)=0.9, P(B)=0.1$. Compare greedy vs beam-2 outputs and probabilities. (Method: Part 12. Greedy: $A \rightarrow B = 0.42$; beam-2 candidates $AA=0.28, AB=0.42, BA=0.27, BB=0.03 \rightarrow$ also $AB=0.42$. They tie — beam can never score below greedy, since greedy's path is always among the beams.)
11. **(6)** For RAG, classify REALM, RAG-Sequence, RAG-Token and FiD by single- vs multi-document evidence, and state the retrieval time complexity. (Method: Part 13.)
12. **(6)** Write the DPO loss and explain, using the optimal-policy identity, why it needs no separate reward model. (Methods: Parts 7.4 and 15.)

— end of exam-focus deep dive —